

One-Pager: Performance Considerations

Aaron Quashnock

Too Many Active Objects

In my game, the difficulty will scale by introducing more targets and game objects as each run continues. One common occurrence in games that use this method of increasing difficulty, such as *Bloons Tower Defense* [1], is frame rate drops and potential crashes from the large amount of game objects requiring large amounts of computing power.

The most common method of handling this performance issue in Unity is through object pooling and by destroying objects to handle memory leaks. According to Unity Documentation, “Object Pooling is a way to optimize your projects and lower the burden that is placed on the CPU when having to rapidly create and destroy new objects” [2]. I can utilize the `ObjectPool` class in my game with common game objects like targets and bullets to improve performance. In addition, implementing safeguards that use the `destroy` method to delete game objects when they are no longer needed can reduce memory leaks. I intend to utilize both these methods to alleviate this performance concern in my game.

Large Textures

Another issue for the performance of my game could stem from the use of unoptimized textures for large amounts of game objects, such as the targets or bullets. In addition to causing long load times on start, rendering high-resolution sprites for many active game objects could contribute to lower frame rates due to large rendering times for each sprite.

To resolve this performance concern, Unity recommends using compressed textures, saying, “compressed textures only use a fraction of the memory bandwidth needed for uncompressed 32-bit RGBA textures” [3]. Unity supports a variety of compression formats for different systems and so the assets I use in my game can be compressed for the system the player uses. For these reasons, I plan to use compressed textures in my game to solve performance concerns with large textures slowing the frame rate and causing lag.

Inefficient Collision Detection

One final performance concern for 2D shooter games is inefficient collision detection, where each of the targets in the game would continuously check for colliding bullets in each frame. While I plan for most bullets in my game to use hit-scan detection collision

such that costly geometric bullet collision detection wouldn't be needed, some bullets in the game have side effects that may require geometric bullet collision. For example, I plan for a fire bullet upgrade to set targets on fire. Targets that are set on fire can spread the fire to other targets when they collide, requiring geometric bullet collision. If my collision detection is insufficient in this case, then I can predict large frame rate drops.

Spatial Partitioning is commonly used to make detecting collisions for many objects more efficient. In a blog, software developer Peer D says, "Spatial partitioning helps by dividing the game world into smaller sections. This way, you only check for collisions between objects in the same section" [4]. In this way, using spatial partitioning in my game can alleviate this performance concern by reducing the number of collision checks needed.

[1] Ninja Kiwi. (2007). Bloons Tower Defense. computer software. Auckland.

[2] https://docs.unity3d.com/ScriptReference/Pool.ObjectPool_1.html

[3] <https://docs.unity3d.com/2017.2/Documentation/Manual/OptimizingGraphicsPerformance.html>

[4] <https://peerdh.com/blogs/programming-insights/efficient-2d-collision-detection-techniques>